

TAE-1

Q.1) Explain various application areas for MongoDB database.

Ans MongoDB is widely used in modern applications due to its flexibility and scalability:-

1) E-commerce Applications
stores products catalogs, user profiles, and orders with flexible schema.

2) Content Management Systems
handles blogs, articles, and multimedia content efficiently.

3) Real-time Analytics
used for dashboards, logs and event tracking.

4) Social Media Applications
manages user posts, comments, likes and relationships.

5) IoT Applications

Stores large volumes of sensor data.

6) Mobile Applications

Synce data efficiently with backend systems.

Q2) How to Create, use and customize MongoDB Configuration Files.

Ans Step 1:- Create Configuration File.

MongoDB uses a config file (usually mongod.conf)

Ex:-

Storage:

dbPath: /data/db

SystemLog:

destination: file

path: /var/log/mongodb/
mongod.log

logAppend: true

net:

port: 27017

bindIp: 127.0.0.1

security:

authorization: enabled

Step 2:- Start MongoDB using config file

mongod --config /path/to/
mongod.conf

Step 3:- Customization options

• Change Port

net:

port: 27018

• Enable Authentication

security:

authorization: enabled

• Set Database Path

Storage:

dbPath: /custom/data/path

- Enable Replication
replication:
replSetName: rs0

Q3) Illustrate various operators and modifiers used in MongoDB.
Ans MongoDB provides different types of operators and modifiers to manipulate and retrieve data.

1) Query Operators:- used to compare values in docs.

1) \$eq $\Rightarrow$ Equal to
ex $\Rightarrow$ {age: { \$eq: 20 }}

2) \$ne $\Rightarrow$ Not equal
ex $\Rightarrow$ {age: { \$ne: 20 }}

3) \$gt $\Rightarrow$ Greater than
ex $\Rightarrow$ {age: { \$gt: 18 }}

4) \$gte $\Rightarrow$ Greater than or equal
ex $\Rightarrow$ {age: { \$gte: 30 }}

5) \$lte $\Rightarrow$ less than or equal
ex $\Rightarrow$ {age: { \$lte: 30 }}

6) \$lt $\Rightarrow$ less than
ex $\Rightarrow$ {age: { \$lt: 20 }}

7) \$in $\Rightarrow$ Matches any value in array
ex $\Rightarrow$ {age: { \$in: [18, 20] }}

8) \$nin $\Rightarrow$ Not in array
ex $\Rightarrow$ {age: { \$nin: [18, 20] }}

2) Logical Operators:- used to combine multiple conditions.

1) \$and :- All conditions must be true

Ex:-

```
db.students.find( {
  $and: [
    { marks: { $gt: 80 } },
    { grade: "A" }
  ]
})
```

2) \$or → Any condition must be true

Ex

```
db.students.find({
  $or: [
    { marks: { $gt: 80 } },
    { grade: "A" }
  ]
})
```

Modifiers :- They are used with update operation like:-

```
db.collection.updateOne()
db.collection.updateMany()
```

1) \$set :- Adds a new field or updates an existing field.

Ex:-

```
db.students.updateOne(
  { name: "Rahul" },
  { $set: { age: 22, city: "Delhi" } }
)
```

2) \$unset :- Removes a field from a document

ex:-

```
db.students.updateOne(
  { name: "Rahul" },
  { $unset: { city: "" } }
)
```

3) \$mul :- Multiplies the value of a field

ex

```
db.products.updateOne(
  { item: "Pen" },
  { $mul: { price: 2 } }
)
```

4) \$rename :- Renames a field

ex

```
db.students.updateOne(
  { name: "Rahul" },
  { $rename: { "marks": "score" } }
)
```

5) \$currentDate :- Sets field to current date/time

```
ex) db.orders.updateOne(
  { item: "Book" },
  { $currentDate: { lastModified: true } }
)
```


TAE-2

Page No.

Date :

Q1) Explain following aggregation pipeline stages.

1) \$Match ⇒ It filters the documents (like SQL where)

ex:- db.students.aggregate([
 { \$match: { marks :
 { \$gt: 60 } } }])

2) \$Group ⇒ Groups documents and performs calculations

ex:- db.students.aggregate([
 {
 \$group: {
 _id: "\$class",
 totalStudents: { \$sum: 1 }
 }
 }])

3) \$sort ⇒ It sorts documents
1 = ascending, -1 = descending

ex:- { \$sort: { marks : -1 } }

4) \$Limit ⇒ It limits the number of documents

ex:- { \$limit : 5 }

5) \$skip ⇒ It skips specified number of documents

ex:- { \$skip : 10 }

6) \$Project ⇒ It selects required fields

ex:- { \$project: { name: 1, marks: 1 } }

7) \$unwind ⇒ Converts array into separate documents

ex:- { \$unwind: "\$hobbies" }

8) \$out ⇒ stores result in a new collection

ex:- { \$out: "resultCollection" }

Q2) Illustrate various replica set states in MongoDB.

Ans A replica set in MongoDB is a group of mongod instances that maintain

the same dataset, providing high availability, redundancy and fault tolerance.

Main Replica Set states

1) Primary:- Only one node in a replica set is PRIMARY. It accepts all write operations.

2) SECONDARY:- Replicates data from PRIMARY. Reads data from oplog and applies changes.

3) ARBITER:- Participates in election but does not store data. Used to maintain odd number of voting members.

4) STARTUP:- Initial state when node starts. Node loads configuration and prepares to join replica set.

5) RECOVERING:- Node is syncing data but cannot accept reads / writes. Happens after restart or failure.

6) Down:- Node is unreachable or crashed.

Q3) Explain how to add an ~~arbitrator~~ in replica set.
arbiter

Ans An arbiter is a MongoDB replica set member that:

- Does not store data
- Does not handle read / write operations
- Participates only in election voting.

Steps to Add an Arbiter

Step 1 ⇒ start MongoDB instance

```
mongod --port 27018 --dbpath /data /arbiter
```


Step 2 ⇒ Connect to primary node

mongo

Step 3 ⇒ Add arbiter

rs.addArb("localhost:27018")

Step 4 ⇒ Verify Configuration

rs.status()

Output will show:

"arbiterOnly" : true

- Arbiter does not store data.
- Helps maintain odd number of nodes
- Prevents split-brain problem
- Cannot become PRIMARY

Q4) Explain the steps involved in export and import of data in MongoDB.

Ans MongoDB provides tool for data migration, backup and transfer.

Export Data (mongoexport)

Purpose:- Convert MongoDB data into JSON or CSV format.

Syntax:-

mongoexport --db=<db-name>
--collection=<collection-name> --out=<file>

Example (JSON Export)

mongoexport --db=school
--collection=students --out=students.json.

Example (csv Export)

```
mongoexport --db=school
--collection=students \
--type=csv --fields=name,
marks \ --out=students.csv
```

Import Data (mongoimport)

Purpose :- Load data into
MongoDB From JSON
/ csv Files

Syntax :-

```
mongoimport --db=<db-name>
--collection=<collection-name>
--file=<file>
```

Example (csv Import)

```
mongoimport --db=school
--collection=students \
--type=csv --file=students
.csv --headerline
```

Example (JSON Import)

```
mongoimport --db=school
--collection=students.json
```

Steps :-

- 1) Select database and collection
- 2) Use mongoexport to export data
- 3) Store or modify the file
- 4) Use mongoimport to import data
- 5) Verify the data